

計算機科学概論

第 6 回

塚田 武志

2025.11.07 千葉大学

今回の目標

計算機を制御する言語を学ぶ

機械語 / アセンブリ

- 基本概念
- 機械語の定義
- 展望

機械語 / アセンブリ

- **基本概念**
- 機械語の定義
- 展望

機械語

計算機の状態を制御する， 計算機が直接解釈できるプログラミング言語

- **簡単な命令**の列

- 例

- メモリの 106 番地に 0 を格納
 - メモリの 1 番地のデータとメモリの 2 番地のデータを足して 3 番地に格納

- 各命令は**固定長の 0/1 の列**

- 演習で扱う CPU においては各命令は 16 bit

ハードウェアとソフトウェアを繋ぐインターフェース

計算機の構成要素

• メモリ

- 計算機がデータを主に格納する場所
- 固定幅のデータ（今回は 16 bit）を記憶できるセルの集まり
- 各セルはユニークなアドレス（今回は 15 bit）を持つ

• レジスタ

- メモリより数が少ない（今回は 2 つ）記憶素子
- アクセスが非常に高速，かつ指定のための情報が少なくて済む
- プロセッサの演算の主な対象

• プロセッサ

- 命令を実行する機器。メモリやレジスタへの読み書きも行う

ニーモニツク / アセンブリ

機械語の命令を人間に分かりやすい記号で書いたもの

- 「1010001100011001」代えて「ADD R1 R3 R2」
- アセンブリは変数が使えるなどの違いもある

プログラムを読み書きするには，機械語よりもアセンブリの方が楽

→ 以降もアセンブリ風の表記で議論します

例：ミンスキー機械

現実の計算機を模した抽象機械

- 記憶素子はレジスタだけ. メモリはない

定義 レジスタが k 個の **ミンスキー機械の命令** は次の 2 種類：

- $\text{INC}(\text{Ri})$ i 番目のレジスタ Ri の値を 1 増やす
- $\text{DEC}(\text{Ri})$ i 番目のレジスタの値を 1 減らす
- $\text{JZ}(\text{Ri}, \text{Z})$ i 番目のレジスタの 0 なら Z 番目の命令に飛ぶ

レジスタが k 個の **ミンスキー機械のプログラム** とは, 上の命令の有限列

ミンスキー機械によるプログラム例

注 左の**命令番号**は便宜的なもの

```
1:  JZ(R1,5)
2:  INC(R2)
3:  DEC(R1)
4:  JZ(R3,1)
```

ミンスキー機械のプログラムの実行

各ステップでひとつの命令を実行し, 次の命令に移る

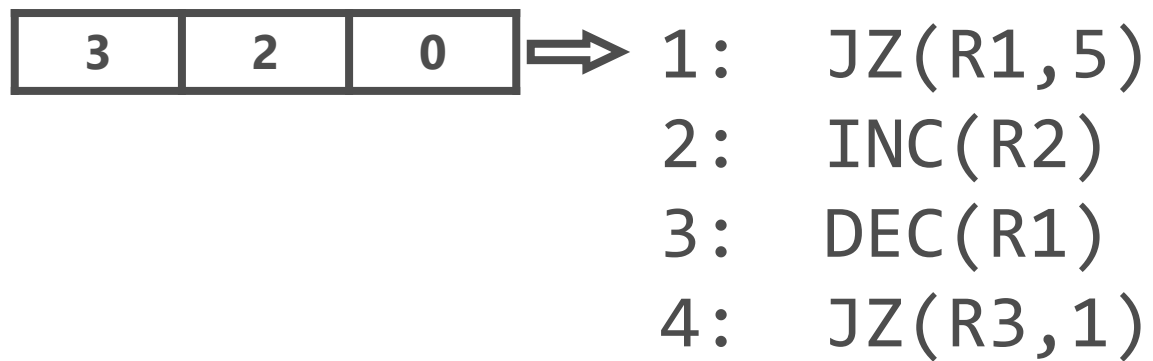
- ただし JZ 命令だけは, 次の命令でなく 指定された命令に移る ことも

実行する命令がないときに終了する

- 最後の命令を実行して, その次の命令に移るとき
- JZ 命令のジャンプ先が存在しない命令だったとき

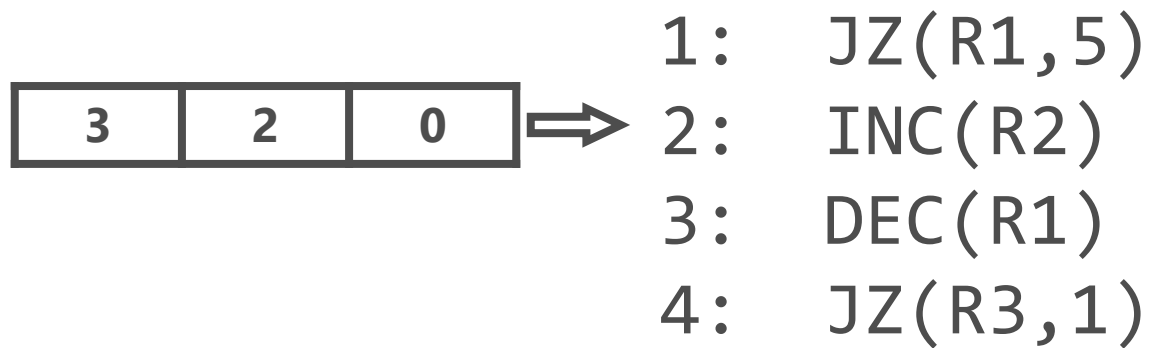
例

注 左の**命令番号**は便宜的なもの



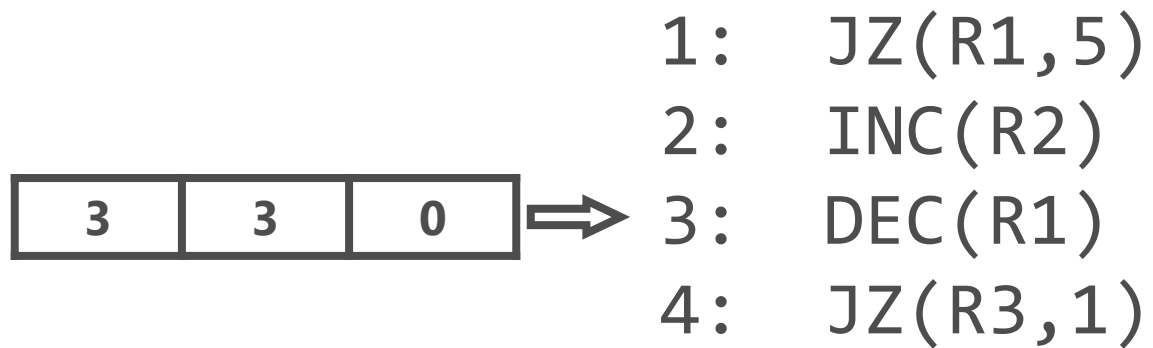
例

注 左の**命令番号**は便宜的なもの



例

注 左の**命令番号**は便宜的なもの



例

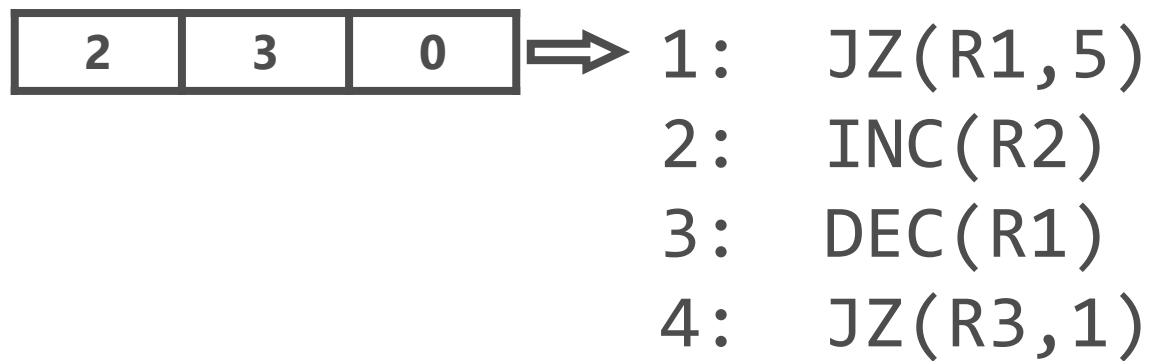
注 左の**命令番号**は便宜的なもの

2	3	0
---	---	---

 $\Rightarrow$
1: JZ(R1,5)
2: INC(R2)
3: DEC(R1)
4: JZ(R3,1)

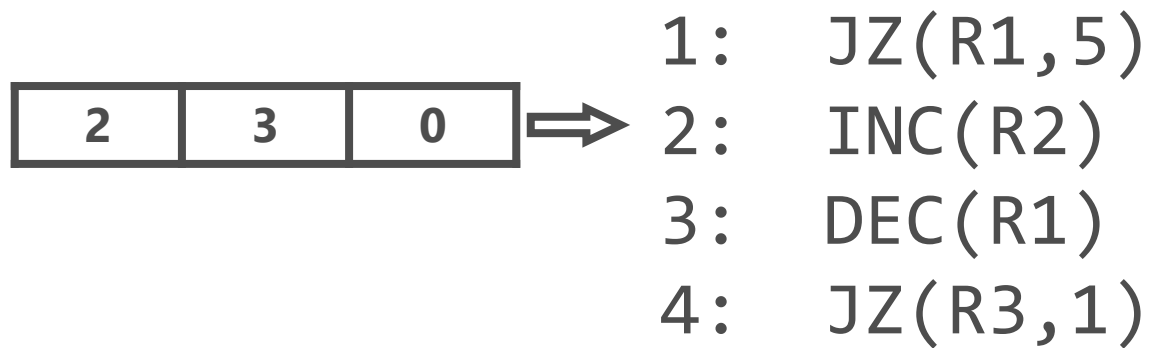
例

注 左の**命令番号**は便宜的なもの



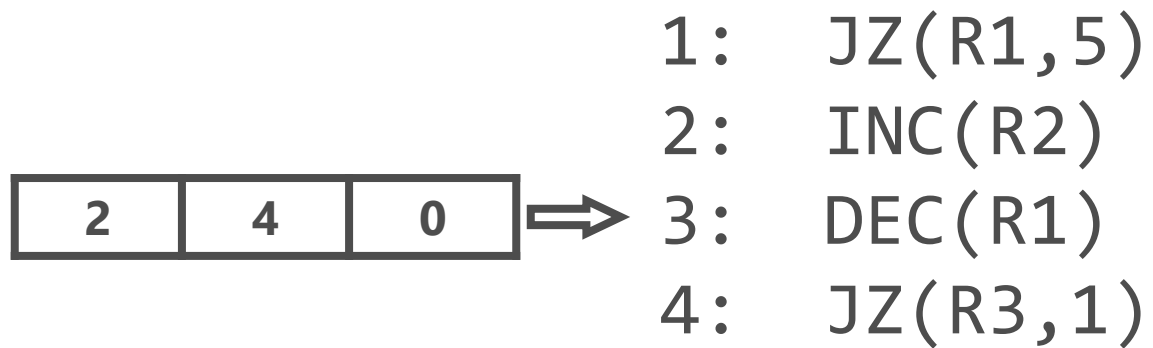
例

注 左の**命令番号**は便宜的なもの



例

注 左の**命令番号**は便宜的なもの



例

注 左の**命令番号**は便宜的なもの

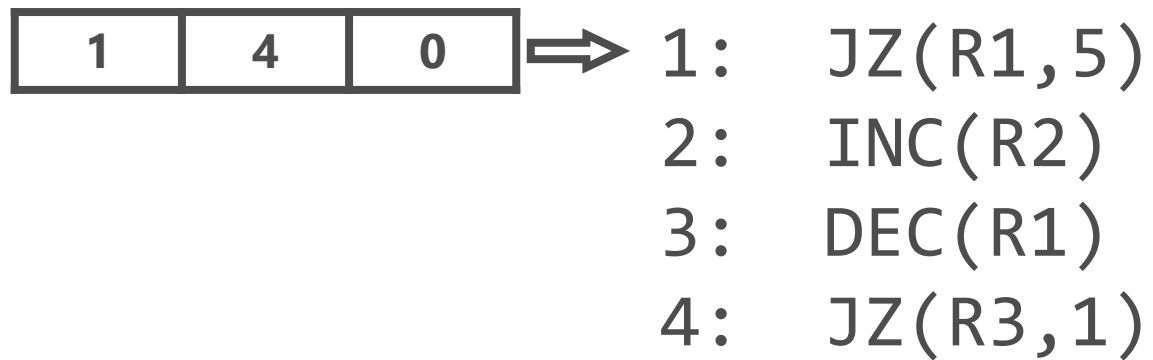
1	4	0
---	---	---

 $\Rightarrow$

1: JZ(R1,5)
2: INC(R2)
3: DEC(R1)
4: JZ(R3,1)

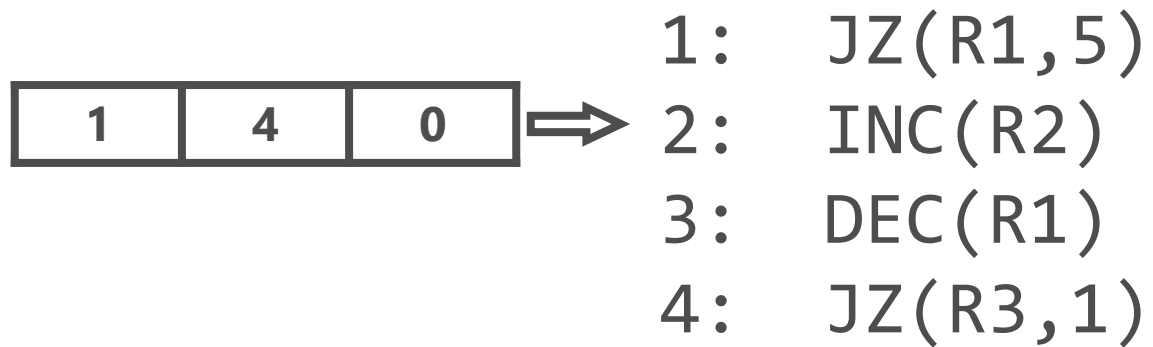
例

注 左の**命令番号**は便宜的なもの



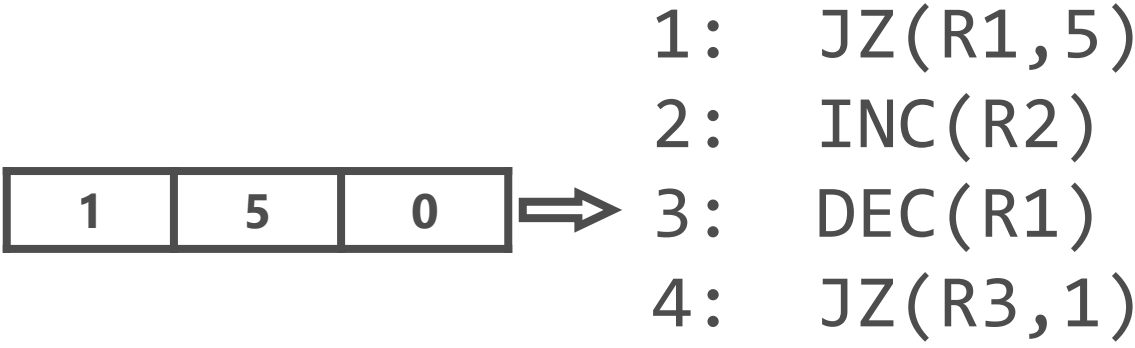
例

注 左の**命令番号**は便宜的なもの



例

注 左の**命令番号**は便宜的なもの



例

注 左の**命令番号**は便宜的なもの

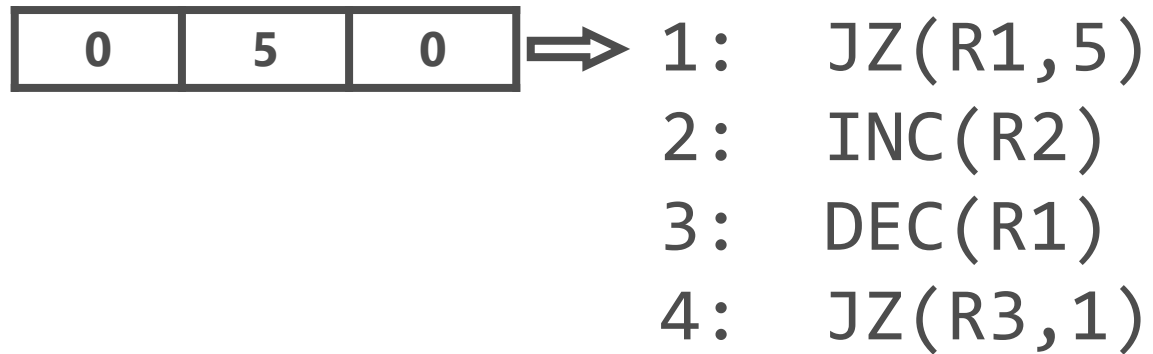
0	5	0
---	---	---

 $\Rightarrow$

1:	JZ(R1, 5)
2:	INC(R2)
3:	DEC(R1)
4:	JZ(R3, 1)

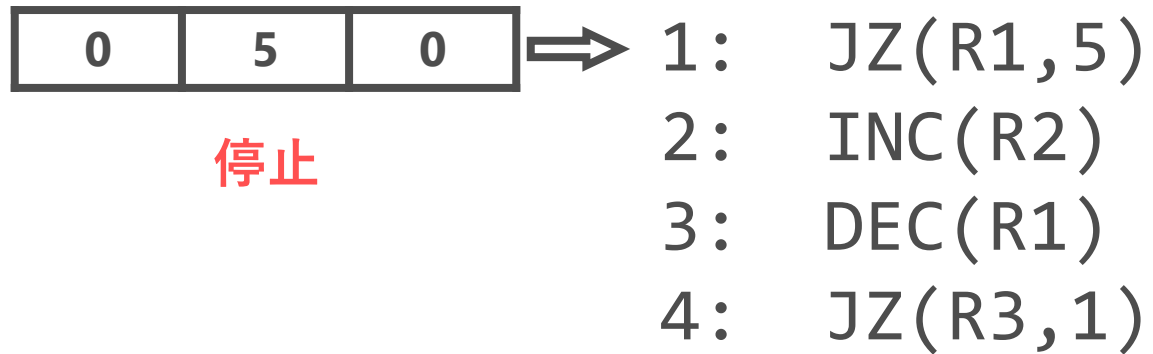
例

注 左の**命令番号**は便宜的なもの



例

注 左の**命令番号**は便宜的なもの



もうちょっと現実寄りの命令たち

$M[R_i]$ でレジスタ R_i の指定する位置のメモリセルを表すとする

- R_i に 2834 が格納されているなら $M[R_i]$ は 2834 番地のセル

現実の計算機の機械語には，次のような感じの命令が備えられている

- `ADD  $R_i$   $R_j$   $R_k$`
- `JMP  $R_i$`
- `LOAD  $R_i$   $M[R_j]$`
- `STORE  $R_i$   $M[R_j]$`
- `ADD  $R_i$   $R_j$   $M[R_k]$`

機械語 / アセンブリ

- 基本概念
- **機械語の定義**
- 展望

演習で使う計算機の構成

注意 一般的な計算機の構成や命令とは異なる面があります

メモリ

- 15 bit アドレスで各セルには 16 bit の情報が格納
- 一部がスクリーンおよびキーボードに直結（**メモリマップド I/O**）

レジスタ

- A レジスタと R レジスタの 2 つ

演習で使う計算機の命令

注意 一般的な計算機の構成や命令とは異なる面があります

- **A命令** Aレジスタに定数値を入れる
- **C命令** 計算を行う

この他に**ラベル定義**がある

A命令

@定数

- A レジスタに指定された値を入れる

例

@100 // A レジスタに 100 が入る

C命令

(**代入先**)=(**式**);(**ジャンプ種別**)

- 次のような一連の計算を行う
 1. まず「**式**」を計算する
 2. 計算結果を「**代入先**」に格納する
 3. 計算結果が「**ジャンプ種別**」の**条件に合っているか**チェック
 - Yes → **Aレジスタで指定された命令に移動**
 - No → **次の命令に移動**

例 D=A+1;JGE

- 「Aレジスタの値 +1」を計算し，Dレジスタに格納
- もし「Aレジスタの値 +1」が正なら，Aレジスタの指示する先に飛ぶ

記号の約束

記憶領域を表す記号たち

- A A レジスタを表す
- D D レジスタを表す
- M メモリの A レジスタで指定された場所を表す
M[A] だと思ふとよい

式

M と A の演算はない
なぜか？

次の式が利用できる

$\emptyset$	$D+1$	M
1	$A+1$	$!M$
-1	$D-1$	$-M$
D	$A-1$	$M+1$
A	$D+A$	$M-1$
$!D$	$D-A$	$D+M$
$!A$	$A-D$	$D-M$
$-D$	$D\&A$	$D\&M$
$-A$	$D A$	$D M$

注意 厳密に上と一致していないとエラー. 例えば $A+D$ や $D \& A$ など

代入先

{A, M, D} の任意の組み合わせ

- 空のときは `null`
- **AMD** の部分文字列でないといけない
 - **DA** は言いたいことは分かるけどエラー

ジャンプ種別

次の 8 種類. いずれの場合も飛び先は A レジスタで指定された場所

- null ジャンプしない
- JGT (計算結果) > 0
- JEQ (計算結果) $= 0$
- JGE (計算結果) ≥ 0
- JLT (計算結果) < 0
- JNE (計算結果) $\neq 0$
- JLE (計算結果) ≤ 0
- JMP 必ずジャンプ

例： $M[4] = M[0] + M[1] + M[2] + M[3]$

@0

D=M;null

@1

D=D+M;null

@2

D=D+M;null

@3

D=D+M;null

@4

M=D;null

C 命令の略記

- 「代入先」が `null` のとき `"(代入先)="` を省略してよい

例 `null=D-1;JLE` は `D-1;JLE` と書ける

- 「ジャンプ種別」が `null` のとき `";null"` を省略してもよい

例 `D=M-1;null` は `D=M-1` と書ける

例： $M[4] = M[0] + M[1] + M[2] + M[3]$

@0

D=M;null

@1

D=D+M;null

@2

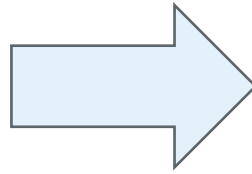
D=D+M;null

@3

D=D+M;null

@4

M=D;null



@0

D=M

@1

D=D+M

@2

D=D+M

@3

D=D+M

@4

M=D

ラベル定義

(LABELNAME)

- LABELNAME という名前のラベルを定義
- **直後の命令の場所**を表す整数値を表す
- **ジャンプ命令の飛び先**を指定するのに利用

例：1 から 100 までの総和

```
@1  
M=0           // M[1] = 0
```

```
@2  
M=0           // M[2] = 0
```

(LOOP) // ループの開始場所のラベル定義

```
@2  
MD=M+1        // M[2] = M[2] + 1; D=M[2]
```

```
@1  
M=M+D         // M[1] = M[1] + D つまり M[1] = M[1] + M[2]
```

```
@2  
D=M
```

```
@100  
D=A-D         // D = 100 - M[2]
```

```
@LOOP  
D;JGT         // if (100 - M[2] > 0) then goto LOOP
```

(END) // ラベル END. プログラムの終了後に悪さをしないように, ここで無限ループ

```
@END  
0;JMP         // goto END
```

変数

ラベルとして定義されていない変数は，適当な値だと解釈される

- 他で利用されていないような場所を選んでくれる

例

@i	// どこかは分からないけど, どこかのアドレス // 変数「i」の格納場所として使おう！
M=M+1	// i = i + 1
D=M	// D = i
@sum	// どこかは分からないけど, iとは違うアドレス
M=D+M	// sum = sum + i

例：1 から 100 までの総和

```
@i
M=0                // i = 0

@sum
M=0                // sum = 0

(LLOOP)            // ループの開始場所のラベル定義

@i
MD=M+1             // i = i + 1;  D = i

@sum
M=M+D               // sum = sum + i   (なぜなら D = i をさっき行ったので)

@i
D=M

@100
D=A-D              // D = 100 - i

@LOOP
D;JGT              // if (100 - M[2] > 0) then goto LOOP

(END)              // ラベル END. プログラムの終了後に悪さをしないように, ここで無限ループ

@END
0;JMP              // goto END
```

メモリマップド I/O

演習で使っている計算機には 2 つの特殊なデバイスがある

- スクリーン
白/黒 が 512×256 並んだもの
メモリのアドレス 16384~24575 の内容を表示
(シンボル SCREEN がスクリーンの領域の先頭のアドレス)
- キーボード
押されているキーを一つ認識できる
メモリのアドレス 24576 にキーの内容が書かれる
なお、押していない = 0
(シンボル KBD が 24576 と定義されている)

例：キーが押されていたら，左上を塗る

@KBD

D=M // いまキーボードの内容を D レジスタに保存

@END

D;JEQ // もし D=0 なら END に飛ぶ

@SCREEN

M=1 // スクリーンの左上を黒 (1) に塗る

(END)

@END

0;JMP

機械語 / アセンブリ

- 基本概念
- 機械語の定義
- **展望**

展望

演習で使っている機械語 / アセンブリは非常に単純

- 多くの機械語は、もっと豊富なメモリアクセス手段を持つ
- 多くの機械語は、もっと豊富な演算を持つ

などなど

演習で使っている機械語 / アセンブリの構文は少し変わっている

- 多くのアセンブリでは **ADD R1 R2** や **LOAD R3 M** などと書く
- ただ、これは表層的な差なので、あまり気にしなくて良い

展望

演習で使っている機械語 / アセンブリでプログラムを書くと

A命令とC命令が交互に現れることが多い

- いくつかのA命令とC命令をセットで指定できる命令（マクロ命令）を用意すると、ぐっと使いやすくなる

例 @xxx を $D=D+M[xxx]$ と略記
 $D=D+M$

アセンブリから機械語への翻訳は、後で実装することになる

課題 6

締切：2025/12/5 23:59 (JST)

課題 6

掛け算をアセンブリで実装せよ

- アセンブリのコードを与え、その説明をせよ

発展 キーが押されているときにスクリーンを黒く、キーが押されていないときにはスクリーンが白くなるようなアセンブリプログラムを書け

- 色塗りに少し時間がかかるので、反応は瞬時でなくてよい
 - なお CPU シミュレータのシミュレーション速度を最大にすると負荷がかかりすぎて動作が不安定になるかもしれない
 - 最速から 2 番目の設定がおすすめ

来週の講義もこの課題に取り組みます